

DSA LinkedList

Complete Pattern Notes
Self-Study & Last-Minute Revision Guide

LinkedList Patterns — Overview

Pattern	Core Idea	Signal Words
Fast & Slow Pointer	Two pointers at different speeds to find cycle, middle, nth node	cycle, middle, nth from end
Reverse	In-place reversal using 3 pointers (prev, curr, next)	reverse, flip, reverse between, reverse in groups
Merge Technique	Merge two or K sorted lists by comparing node values	merge sorted lists, merge K lists
Dummy Head Node	Sentinel node before head to simplify edge cases	deletion, insertion at head, head might change
Multiple Pass	First pass gathers info, second pass acts on it	reorder, rearrange, palindrome check

LinkedList problems combine multiple patterns. Reorder List = find middle + reverse + merge. Palindrome = find middle + reverse + compare.

Pattern 1 — Fast & Slow Pointer

Core Idea: Two pointers moving at different speeds (fast moves 2 steps, slow moves 1). Used to detect cycles, find midpoints, and maintain gaps.

The 3 Variants

Variant	Fast Speed	Slow Speed	Use For
Basic	2 steps	1 step	Detect cycle, find middle
With gap of n	2 steps (after offset)	1 step	Find nth node from end
Reset pointer	1 step (after meeting)	1 step (from head)	Find cycle start

Find Cycle Start — The Math (Leetcode 142)

After slow and fast meet inside cycle — reset one pointer to head and move both at speed 1. They meet exactly at cycle start.

```
Let a = distance head to cycle start
Let b = distance cycle start to meeting point
Let c = remaining cycle length

slow traveled: a + b
fast traveled: a + b + b + c (went around cycle once extra)
fast = 2 * slow -> a + 2b + c = 2a + 2b -> c = a

Conclusion: distance from meeting point back to cycle start
            EQUALS distance from head to cycle start.

ListNode* detectCycle(ListNode* head){
    ListNode *slow=head, *fast=head;
    while(fast && fast->next){
        slow=slow->next; fast=fast->next->next;
        if(slow==fast){
            ListNode* ptr=head;
            while(ptr!=slow){ ptr=ptr->next; slow=slow->next; }
            return ptr;
        }
    }
    return nullptr;
}
```

Nth From End — Gap Technique (Leetcode 19)

Move fast n steps ahead first. Then move both together — maintains exactly n gap. When fast reaches end, slow is at the right position.

```
ListNode dummy(0); dummy.next = head;
ListNode *slow=&dummy, *fast=&dummy;
```

```

for(int i=0; i<n; i++) fast=fast->next;    // create gap of n
while(fast->next){ slow=slow->next; fast=fast->next; }
slow->next = slow->next->next;              // remove nth from end
return dummy.next;

```

Why dummy node here: if we remove the head itself (n equals list length), dummy node handles it cleanly without special casing.

Solved Problems

Problem	LC#	Key Insight
Linked List Cycle	141	slow and fast meet -> cycle exists
Linked List Cycle II	142	After meeting, reset to head. c=a guarantees they meet at cycle start
Middle of Linked List	876	When fast reaches end, slow is at middle
Remove Nth From End	19	Gap of n between fast and slow, use dummy head

Pattern 2 — Reverse

Core Idea: In-place reversal using 3 pointers (prev, curr, next). Save next before breaking the link.

Base Template — Full Reversal

```
ListNode* reverseList(ListNode* head){
    ListNode *prev=nullptr, *curr=head;
    while(curr){
        ListNode *next = curr->next;    // ALWAYS save next first
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;    // prev is new head
}
```

The #1 bug in reversal: forgetting to save curr->next before overwriting curr->next. Always save next first.

Reverse Between Positions (Leetcode 92)

The insert-at-front trick: Instead of standard 3-pointer reversal — repeatedly take the next node and insert it right after prev. Naturally reverses the segment.

```
ListNode dummy(0); dummy.next=head;
ListNode* prev=&dummy;
for(int i=0; i<left-1; i++) prev=prev->next;    // reach node before reversal
ListNode* curr=prev->next;
for(int i=0; i<right-left; i++){
    ListNode* next=curr->next;
    curr->next=next->next;
    next->next=prev->next;
    prev->next=next;
}
return dummy.next;
```

Reverse in Groups of K (Leetcode 25)

Key steps: (1) Count K nodes — if less than K remain, return head unchanged. (2) Recursively solve the rest first. (3) Reverse current K nodes, connecting to recursive result.

```
ListNode* reverseKGroup(ListNode* head, int k){
    ListNode* curr=head;
    int count=0;
    while(curr && count<k){ curr=curr->next; count++; }
    if(count<k) return head;    // not enough nodes, leave unchanged
    ListNode* prev=reverseKGroup(curr,k);    // handle rest first
    curr=head;
```

```

while(count--){
    ListNode* next=curr->next;
    curr->next=prev;
    prev=curr; curr=next;
}
return prev;
}

```

Recursion vs iteration: recursion is cleaner here but uses $O(n/k)$ stack space. For strict $O(1)$ space — use iterative approach tracking group connections manually.

Recursion vs Iteration for Reversal

	Recursive	Iterative
Code clarity	Cleaner, natural	More pointer juggling
Space	$O(n)$ stack space	$O(1)$ space
Interview	Explain tradeoff	Preferred if space matters

Solved Problems

Problem	LC#	Key Insight
Reverse Linked List	206	Basic 3-pointer reversal, return prev
Reverse Linked List II	92	Insert-at-front trick, dummy head
Reverse Nodes in k-Group	25	Count K first, recurse rest, then reverse

Pattern 3 — Merge Technique

Core Idea: Compare node values from two lists, attach the smaller one, advance that pointer. Repeat until both lists exhausted.

Base Template — Merge Two Sorted Lists

```
ListNode* mergeTwoLists(ListNode* l1, ListNode* l2){
    ListNode dummy(0);
    ListNode* tail=&dummy;
    while(l1 && l2){
        if(l1->val <= l2->val){ tail->next=l1; l1=l1->next; }
        else { tail->next=l2; l2=l2->next; }
        tail=tail->next;
    }
    tail->next = l1 ? l1 : l2;    // attach remaining
    return dummy.next;
}
```

Merge K Sorted Lists (Leetcode 23)

Two approaches:

Approach	Time	How
Merge one by one	$O(k*n)$	Merge list1+list2, result+list3, etc.
Divide and Conquer	$O(n \log k)$	Pair up and merge in rounds — like merge sort
Min Heap	$O(n \log k)$	Always extract minimum from heap of k heads

Divide and Conquer Code

```
while(lists.size() > 1){
    vector<ListNode*> merged;
    for(int i=0; i<lists.size(); i+=2){
        ListNode* l1=lists[i];
        ListNode* l2=(i+1<lists.size()) ? lists[i+1] : nullptr;
        merged.push_back(mergeTwoLists(l1, l2));
    }
    lists=merged;
}
return lists[0];
```

Min Heap Code

```
priority_queue<pair<int,ListNode*>,
    vector<pair<int,ListNode*>>, greater<>> > pq;
for(auto node : lists) if(node) pq.push({node->val, node});
```

```

ListNode dummy(0); ListNode* tail=&dummy;
while(!pq.empty()){
    auto [val, node] = pq.top(); pq.pop();
    tail->next=node; tail=tail->next;
    if(node->next) pq.push({node->next->val, node->next});
}
return dummy.next;

```

Divide and Conquer is cleaner and doesn't need heap. Min Heap is more intuitive for some and connects to the Heap topic's K-way merge pattern.

Solved Problems

Problem	LC#	Key Insight
Merge Two Sorted Lists	21	Dummy head, compare and attach smaller, attach remaining at end
Merge k Sorted Lists	23	Divide and conquer in log k rounds, or min heap

Pattern 4 — Dummy Head Node

Core Idea: Create a sentinel node before head. Eliminates special-casing when head itself might change (deletion at head, insertion before head).

When to Always Use Dummy Head

Situation	Why Dummy Helps
Deleting the head node	dummy.next = head; after deletion, return dummy.next safely
Inserting before head	prev always has a valid node, even at position 0
Merging lists (result head unknown)	Dummy avoids special-casing which list starts the result
Reverse between positions	prev node before reversal zone always exists

```
ListNode dummy(0);
dummy.next = head;
ListNode* curr = &dummy;
// ... do operations ...
return dummy.next; // actual head (may have changed)
```

Rule of thumb: if your solution needs to handle 'what if the head is removed/changed' as a special case — add a dummy head and the special case disappears.

Pattern 5 — Multiple Pass Technique

Core Idea: First pass gathers necessary information (length, middle, structure). Second (and third) pass acts on that information. Don't try to do everything in one pass when it complicates logic.

Reorder List — 3 Pass Example (Leetcode 143)

Problem: Reorder L0->L1->...->Ln to L0->Ln->L1->Ln-1->...

Pass	Action	Technique Used
Pass 1	Find middle of list	Fast/Slow pointer
Pass 2	Reverse second half	3-pointer reversal
Pass 3	Merge first and reversed half alternately	Two pointer merge

```
// Pass 1: find middle
ListNode *slow=head, *fast=head;
while(fast->next && fast->next->next){ slow=slow->next; fast=fast->next->next; }
ListNode* second=slow->next; slow->next=nullptr;

// Pass 2: reverse second half
ListNode *prev=nullptr, *curr=second;
while(curr){ ListNode* next=curr->next; curr->next=prev; prev=curr; curr=next; }

// Pass 3: merge alternately
ListNode *first=head; second=prev;
while(second){
    ListNode *tmp1=first->next, *tmp2=second->next;
    first->next=second; second->next=tmp1;
    first=tmp1; second=tmp2;
}
```

Palindrome Linked List — 3 Pass Pattern

Same 3-pass structure: find middle, reverse second half, compare both halves.

Pass	Action
Pass 1	Find middle using fast/slow pointer
Pass 2	Reverse second half in place
Pass 3	Compare first half and reversed second half node by node

When Multiple Pass Is Needed

Signal	Pattern
reorder, rearrange list structure	3 pass: find middle + reverse + merge

Signal	Pattern
palindrome check on linked list	3 pass: find middle + reverse + compare
remove nth from end (one pass variant)	2 pointer gap — effectively 1 pass
need length first, then act on index	2 pass: count length, then traverse to position

Solved Problems

Problem	LC#	Passes Used
Reorder List	143	Find middle + reverse second half + merge alternately
Palindrome Linked List	234	Find middle + reverse second half + compare

Quick Reference — All 5 LinkedList Patterns

Pattern	Time	Space	Use When
Fast & Slow Pointer	$O(n)$	$O(1)$	Cycle, middle, nth from end
Reverse	$O(n)$	$O(1)$ iterative / $O(n)$ recursive	Flip direction, groups, segments
Merge Technique	$O(n \log k)$	$O(1)$ D&C / $O(k)$ heap	Combining sorted lists
Dummy Head Node	$O(1)$	$O(1)$	Head might change, edge cases
Multiple Pass	$O(n)$	$O(1)$	Reorder, palindrome, complex rearrangement

Most Important Rules

- Fast/Slow: after meeting point in cycle — reset one to head, move both at speed 1. They meet at cycle start (c=a proof).
- Reverse: ALWAYS save next = curr->next BEFORE overwriting curr->next. Missing this is the #1 bug source.
- Reverse K-Group: count K nodes FIRST — if count < K, return head unchanged without reversing.
- Merge K lists: divide and conquer ($O(n \log k)$) beats merge one-by-one ($O(k*n)$). Pair up in rounds.
- Dummy Head: use whenever head itself might change — eliminates all special cases at position 0.
- Multiple Pass: don't force everything into one pass — find middle, reverse, merge are cleaner as 3 separate passes.
- Recursion vs Iteration: recursion is cleaner but $O(n)$ stack space. Iterative is $O(1)$ space but more complex.

— End of Notes —